EXP-Bench: Can AI Conduct AI Research Experiments?

Taro Yamada

Abstract

Benchmarks for language agents have made substantial progress on coding, web interaction, and machine learning engineering, but they still under-measure whether an agent can carry out a complete AI research experiment from a paper-grounded question to an executable implementation and a justified conclusion. We introduce EXP-Bench, a benchmark for end-to-end AI experimentation built from peer-reviewed AI papers and their public repositories. EXP-Bench contains 461 tasks from 51 NeurIPS and ICLR 2024 papers, totaling 12,737 fine-grained subtasks across computer vision, natural language processing, reinforcement learning, and application domains. Each task provides a research question, a high-level method description, and a masked repository, while the benchmark stores execution-validated ground truth for experimental design, code changes, and expected conclusions. Evaluation combines judged scores for design, implementation, and conclusion with containerized execution checks and conjunctive metrics that require multiple stages of the workflow to be correct simultaneously. Across seven agent-model configurations, current systems show non-trivial partial competence but almost never complete the full workflow reliably: the strongest implementation and execution scores come from OpenHands with Claude-3.7 Sonnet at 35.0% and 33.2%, yet the strict end-to-end metric All·E✓ remains 0.4%; OpenHands with o3-mini reaches the highest judged conclusion score at 21.0% and the highest All✓ at 1.4%, but only 0.5% on All·E✓. These results show that EXP-Bench is challenging, execution-grounded, and diagnostically useful for measuring where contemporary AI agents fail in realistic research experimentation workflows.

1 Introduction

AI research experimentation is an unusually demanding agent task. A competent system must identify the experimentally relevant variables from a paper, modify code in the correct locations, execute the result in a working environment, and state a conclusion that matches what was actually run. Existing evaluations typically isolate only part of this workflow: code benchmarks emphasize local program synthesis, general agent benchmarks emphasize interaction and planning, and scientific-agent benchmarks often focus on narrower forms of code generation or discovery tasks rather than paper-grounded experimental replication [7, 3, 2, 1, 8].

EXP-Bench targets this gap. The benchmark asks whether an AI agent can conduct a concrete AI research experiment anchored in a recent peer-reviewed paper and its public codebase. Each task begins from a realistic experimental question extracted from the paper, provides a partially masked repository to prevent shortcutting, and requires the agent to produce a design, implementation, execution, and conclusion. This framing evaluates end-to-end experimental competence rather than isolated coding ability.

The benchmark is motivated by two design principles. First, correctness should be grounded in executable artifacts whenever possible. A judged plan or plausible patch is not sufficient if the resulting experiment does not run or support the claimed conclusion. Second, evaluation should

Preprint. Under review.

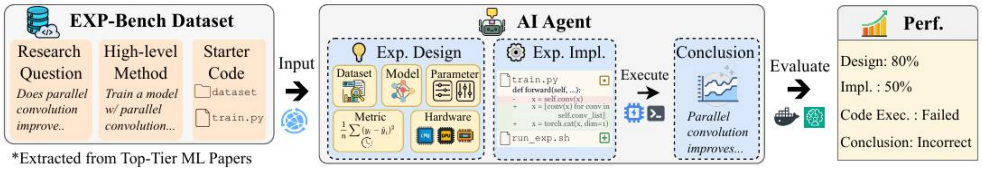


Figure 1: Overview of EXP-Bench. Tasks are extracted semi-autonomously from peer-reviewed AI papers and their repositories, then used to evaluate whether an agent can complete a full research experiment rather than only a local coding subproblem.

remain diagnostic. Rather than collapsing performance into a single scalar, EXP-Bench separates design (D), implementation (I), execution (E), and conclusion (C), and then reports conjunctive metrics that expose how quickly apparent competence disappears when multiple stages must be correct at once.

Using the released benchmark artifacts, we evaluate seven agent-model configurations. The main empirical result is stark: contemporary agents often solve fragments of the workflow, but almost never complete the full loop. For example, OpenHands with Claude-3.7 Sonnet achieves the strongest implementation and execution scores, but only 0.4% on the strict end-to-end metric All·E✓. OpenHands with o3-mini reaches the best judged conclusion score and the best judged all-stage score, but still only 0.5% on All·E✓. A category breakdown further shows that reinforcement learning tasks are relatively more tractable than application-oriented tasks, yet even there end-to-end completion remains rare.

This paper makes four contributions. First, it introduces EXP-Bench, a benchmark of 461 executable AI research experiment tasks from 51 NeurIPS/ICLR 2024 papers. Second, it describes a semi-automated construction pipeline that extracts task statements and implementation requirements from papers and repositories, then validates them through execution and final human checks. Third, it proposes an evaluation protocol that combines LLM-based judging, execution validation, and integrity monitoring with strict conjunctive metrics. Fourth, it provides an empirical analysis showing that current agentic systems remain far from reliable end-to-end AI experimentation.

2 Related Work

General agent benchmarks. Recent work has expanded evaluation from static QA to interactive agent settings. AgentBench evaluates decision making across diverse environments [7], while platforms such as OpenHands emphasize general software-agent interaction through terminals, code editing, and browsing [9]. These benchmarks are valuable for measuring tool use and sequential reasoning, but they do not directly test whether an agent can reproduce a realistic AI research experiment grounded in a paper and its codebase.

Machine learning and scientific experimentation benchmarks. Benchmarks closer to our target setting include MLAgentBench, which studies machine learning experimentation tasks such as model improvement and challenge solving [3]; MLE-bench, which evaluates machine learning engineering on Kaggle-style competitions [1]; ScienceAgentBench, which extracts scientific coding tasks from publications across multiple disciplines [2]; and PaperBench, which evaluates replication of recent AI papers through rubric-based grading [8]. EXP-Bench is complementary to these efforts. Compared with MLAgentBench and MLE-bench, it is more tightly anchored to paper-specific experimental procedures and conclusions. Compared with ScienceAgentBench, it focuses specifically on AI research experiments and uses masked repositories plus execution-grounded validation. Compared with PaperBench, it evaluates experiment completion with explicit execution checks and stagewise conjunctive metrics, rather than a single rubric score.

AI systems for scientific research. Method papers increasingly claim progress toward automated research, including systems for machine learning experimentation and broader scientific assistance [5, 10, 4]. Those systems increase the need for evaluation frameworks that distinguish partially correct research behavior from reliable end-to-end experimentation. EXP-Bench is intended as such

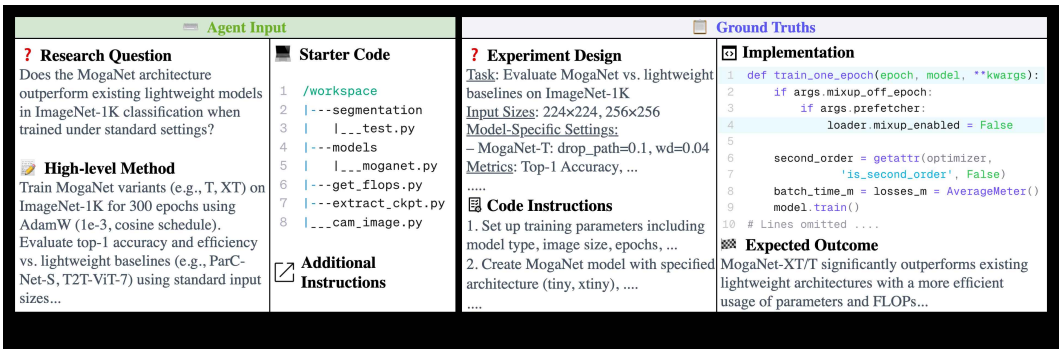


Figure 2: Example EXP-Bench task based on the MogaNet repository [6]. The benchmark supplies a paper-grounded research question and masked repository state, then evaluates the agent on design, implementation, execution, and conclusion.

an evaluation instrument: it measures a specific, operationally defined slice of scientific competence and does not claim to capture the full scope of scientific discovery.

3 Benchmark Design

3.1 Task formulation

EXP-Bench is built around AI research experiment tasks derived from recent peer-reviewed papers and their public repositories. The benchmark contains 461 tasks from 51 NeurIPS and ICLR 2024 papers, totaling 12,737 fine-grained subtasks. Domains include computer vision, natural language processing, reinforcement learning, and applications. Figure 3b shows that the resulting task set spans a diverse collection of research categories rather than a single subfield.

Each task exposes three ingredients to the evaluated agent: a research question, a high-level description of the relevant method, and a masked repository. The masking step removes answer-revealing files such as selected README content or scripts that would otherwise trivialize the task. Ground truth stores three aligned artifacts: a design specification describing the required experiment, a git-diff implementation describing the necessary code changes, and a validated conclusion describing the expected experimental outcome.

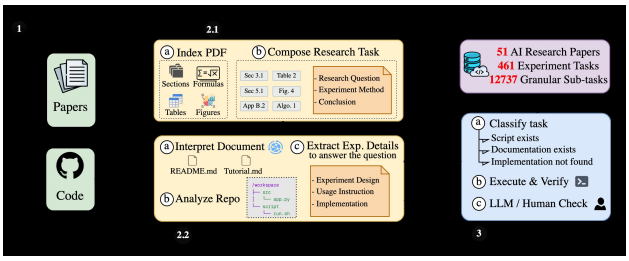
This design deliberately evaluates a workflow rather than a single output. A correct answer can fail because the design omits an essential variable, because the patch changes the wrong files, because the environment does not execute, or because the final conclusion misstates the result. EXP-Bench keeps these failure modes visible.

3.2 Metrics and protocol

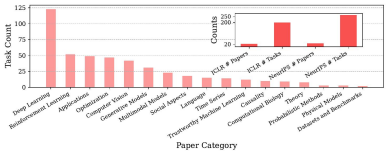
EXP-Bench uses four primary metrics. Design (D) measures whether the agent identifies the correct experimental variables, procedure, and constraints. Implementation (I) measures whether the code modifications capture the required changes. Execution (E) measures whether the produced experiment executes successfully under the benchmark validator. Conclusion (C) measures whether the final written conclusion correctly reflects the experimental outcome.

According to the benchmark specification, D, I, and C are graded by an o3-mini LLM judge, while E is determined by an execution validator. The benchmark additionally reports stricter conjunctive metrics: I·E requires both implementation and execution to be correct; All✓ requires D, I, and C to all be correct; and All·E✓ requires correctness on all four stages. These conjunctive metrics are central to the benchmark because they separate local progress from usable experimental completion.

To preserve benchmark integrity, evaluation is run with a monitor that flags disallowed behavior, including attempts to access the paper directly, revert the repository state through forbidden git operations, or fabricate outputs such as fake data. Execution uses Dockerized environments with Ubuntu 24.04, up to four A40 GPUs, and a 40-minute timeout per run.



(a) EXP-Bench semi-automated construction pipeline.



(b) Category distribution of evaluated tasks.

Figure 3: Benchmark construction and coverage. EXP-Bench combines paper parsing, repository analysis, execution-based refinement, and final human checks to produce a diverse set of AI research experiment tasks.

4 Construction and Validation

4.1 Semi-automated construction pipeline

Figure 3a summarizes the benchmark construction pipeline. Source papers are selected from NeurIPS and ICLR 2024 using influence signals together with the availability of public repositories. Candidate tasks are then extracted from PDFs via OCR and multimodal parsing of figures and tables. Retrieval-augmented, multi-pass prompting assembles structured descriptions of the research question, datasets, metrics, and constraints, with follow-up retrieval used to recover missing details.

The repository side of the task is processed separately. A tool-augmented extraction agent identifies scripts, dependencies, checkpoints, and command chains needed to run the relevant experiment. Those scripts are then traced into step-by-step implementation requirements so that the benchmark can align the intended experimental design with concrete code edits.

4.2 Quality control and benchmark validity

Candidate tasks are validated by executing the extracted scripts in clean containers and comparing the observed behavior against paper- and repository-level expectations. When execution fails or extracted task descriptions are inconsistent, the pipeline iterates on the structured task representation. Final benchmark entries also receive lightweight human review.

This quality-control process provides a pragmatic notion of validity: benchmark instances are not accepted solely because a text extractor or judge produced a plausible description; they must also correspond to a runnable experimental workflow. At the same time, EXP-Bench does not present formal inter-annotator agreement statistics or a fixed annotator pool in the available artifacts. Its main validity evidence therefore comes from execution-based verification, iterative refinement, human cross-checking, and post hoc stability analyses.

The benchmark assets also include a stability analysis (Figure 4), which is consistent with the intended role of the judge as a scalable scoring component rather than a substitute for execution. Taken together, these checks support the claim that EXP-Bench is meaningful and non-trivial, while also clarifying that judged components remain an approximation and should be interpreted alongside E and the conjunctive metrics. These checks also define the scope of the benchmark's validity. EXP-Bench verifies whether an agent can reconstruct and run a paper-grounded experiment under the released repository constraints, not whether the underlying paper claim generalizes beyond that setting. Near-miss outputs can still reflect useful partial understanding, but the protocol scores them as failures when they do not complete the specified workflow.

5 Experimental Setup

We report the seven agent-model configurations present in the released benchmark tables. The evaluated agent scaffolds are OpenHands and IterativeAgent; the evaluated models are o3-mini, Claude-3.7 Sonnet, Claude-3.5 Haiku, Amazon Nova Pro, and DeepSeek R1, depending on the

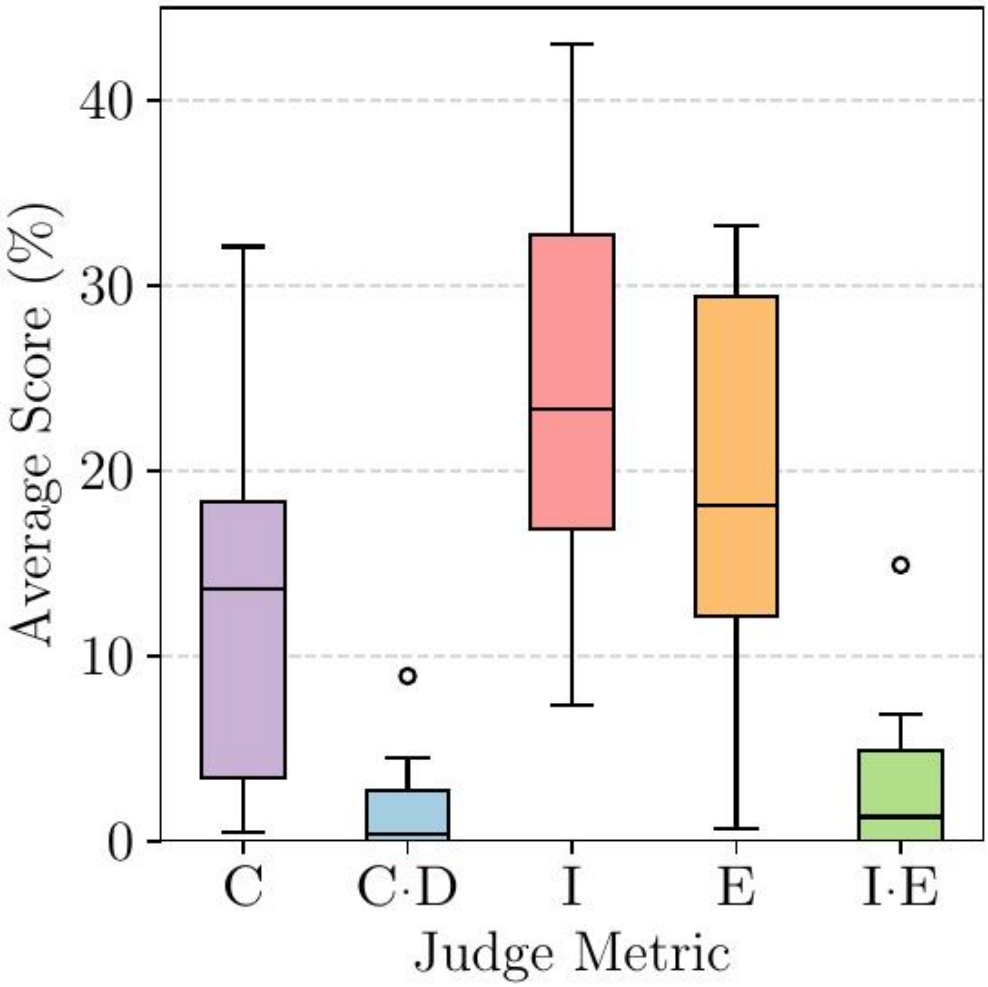


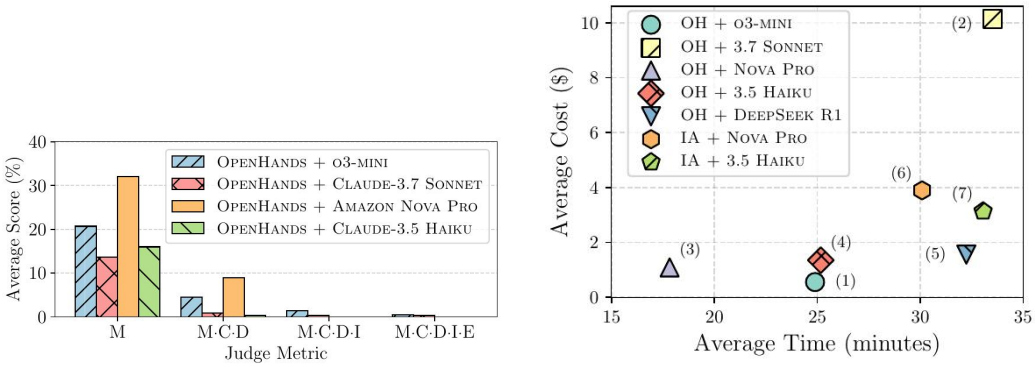
Figure 4: Stability analysis provided with the benchmark artifacts. The figure supports the claim that aggregate conclusions are not driven by a single unstable judge decision or isolated scoring noise.

scaffold. Table 1 reports the average metric values together with #E, the number of execution-scored runs for that configuration.

The benchmark uses a common execution environment based on Docker containers running Ubuntu 24.04, with up to four A40 GPUs and a 40-minute timeout. D, I, and C are judged automatically, while E depends on actual execution under the benchmark validator. Because not every run reaches an execution-scored state, #E varies across configurations and should be considered when comparing E-related columns.

Table 1: Average benchmark scores for various models when tested against various evaluation metrics. Popular Agents and LLMs perform poorly on EXP-Bench, showcasing its difficulty.

Agent	Model	D	I	E	I·E	C	All✓	All·E✓	#E
OpenHands	o3-mini	18.4	20.3	15.0	2.9	**21.0**	**1.4**	**0.5**	420
OpenHands	Claude-3.7 Sonnet	16.0	**35.0**	**33.2**	**14.9**	13.4	0.7	0.4	235
OpenHands	Amazon Nova Pro	18.2	19.5	26.8	0.0	15.7	0.0	0.0	56
OpenHands	Claude-3.5 Haiku	**20.6**	26.2	9.3	1.3	13.8	0.0	0.0	237
OpenHands	DeepSeek R1	6.8	10.0	0.7	0.0	2.4	0.0	0.0	140
IterativeAgent	Claude-3.5 Haiku	6.4	20.6	25.2	5.4	2.2	0.0	0.0	111
IterativeAgent	Amazon Nova Pro	0.1	10.0	18.1	0.0	0.3	0.0	0.0	215



(a) Stricter metrics sharply reduce measured performance.

(b) Cost-time trade-offs across agents and metrics.

Figure 5: Ablations included with the benchmark artifacts. EXP-Bench reveals that stronger apparent performance under lenient metrics does not translate into reliable end-to-end correctness, and that higher cost or longer runtime does not by itself guarantee success.

6 Results and Analysis

6.1 Main benchmark results

Table 1 shows that all tested systems perform poorly on strict end-to-end evaluation. OpenHands with Claude-3.7 Sonnet is the strongest configuration on implementation and execution, reaching 35.0 on I, 33.2 on E, and 14.9 on I·E. OpenHands with o3-mini attains the highest conclusion score, 21.0, and the best judged all-stage score, 1.4 on All✓. Yet no configuration exceeds 0.5 on the strictest metric, All·E✓.

This gap between marginal and conjunctive scores is the central empirical finding of EXP-Bench. Systems can often produce some plausible design text, some code modifications, or some partially correct interpretation. What they rarely do is align all stages into a single correct experimental workflow. Averaged across systems, the benchmark overview reports that performance drops from 20.6% under a lenient monitor-pass view to only 0.2% when design, implementation, execution, and conclusion must all be correct jointly. Figure 5a visualizes this effect.

The table also shows that system ranking depends on which stage is measured. OpenHands with Claude-3.7 Sonnet leads on I, E, and I·E, whereas OpenHands with o3-mini leads on C and All✓. This is useful diagnostic signal rather than leaderboard noise: some agents are better at producing executable repository changes but weaker at stating the right takeaway, while others generate more plausible conclusions without reliably completing the underlying experiment. Reporting only a single aggregate score would hide these differences. EXP-Bench instead exposes whether progress reflects genuinely improved experimental reliability or merely a redistribution of errors across planning, coding, execution, and interpretation.

Table 2: Average benchmark scores across representative task categories on EXP-Bench. Scores vary by domain, but end-to-end success remains low even in relatively stronger categories such as RL.

Category	Agent	Model	D	I	E	I·E	C	All✓	All·E✓
Applications	OH	Nova Pro	**19.2**	23.9	19.0	0.0	**13.9**	0.0	0.0
Applications	OH	o3-mini	9.0	8.0	0.0	0.0	8.3	0.0	0.0
Applications	OH	3.5 Haiku	**19.2**	24.5	8.3	5.6	8.3	0.0	0.0
Applications	OH	3.7 Sonnet	9.0	**26.8**	**30.8**	**7.7**	8.3	**2.8**	0.0
Applications	IA	Nova Pro	0.0	9.8	5.0	0.0	0.0	0.0	0.0
Applications	IA	3.5 Haiku	0.0	18.0	0.0	0.0	0.0	0.0	0.0
Applications	OH	DeepSeek R1	3.1	4.3	0.0	0.0	0.0	0.0	0.0
RL	OH	3.7 Sonnet	18.3	**48.2**	**27.3**	**21.2**	17.6	2.0	**3.0**
RL	OH	o3-mini	23.5	34.8	15.7	2.0	**27.5**	**3.9**	0.0
RL	OH	3.5 Haiku	**27.7**	41.4	11.5	0.0	17.6	0.0	0.0
RL	IA	3.5 Haiku	3.3	27.5	17.4	0.0	2.4	0.0	0.0
RL	OH	DeepSeek R1	5.0	10.3	0.0	0.0	2.0	0.0	0.0
RL	IA	Nova Pro	0.0	8.6	9.7	0.0	0.0	0.0	0.0
RL	OH	Nova Pro	17.9	28.9	0.0	0.0	13.7	0.0	0.0

Table 3: Agents fail in diverse ways across different phases of experimentation; this table presents a simplified subset of common examples, measured across all agent and model evaluations.

Phase	Failure Type	Prevalence (%)
Design	Incomplete or Misclassified Design Variables	16.05
Design	Irrelevant Procedural Additions in Design	7.62
Implementation	Missing Essential implementation Components	39.71
Implementation	Incomplete Evaluation Metric Implementation	2.15
Implementation	Incomplete Data and Preprocessing Setup	1.83
Execution	Environment/Dependency Configuration Errors	29.38
Execution	Execution Script and File Errors	23.84
Execution	Missing Setup Script File	6.95
Execution	Tensor Operation Execution Error	3.22
Conclusion	Missing Conclusion Content	26.18
Conclusion	Incorrect Conclusion Interpretation	19.66
Conclusion	Extraneous Details in Conclusion	7.77
Conclusion	Incorrect Numeric Conclusion	3.21

6.2 Performance across task categories

Category-level results in Table 2 show that benchmark difficulty is uneven but consistently high. Reinforcement learning tasks are relatively more tractable: OpenHands with Claude-3.7 Sonnet reaches 48.2 on I and 21.2 on I·E, while OpenHands with o3-mini reaches 27.5 on C and 3.9 on All✓. Even so, the strict end-to-end metric remains only 3.0 at best in this slice. Application tasks are generally harder, with lower D and C scores and almost no fully correct end-to-end runs.

These results are informative rather than merely low. They suggest that some research domains expose more reusable implementation patterns, while others require more accurate interpretation of paper-specific procedural details. EXP-Bench therefore distinguishes among agent capabilities in a way that a single aggregate coding score would miss.

6.3 Failure modes

Table 3 summarizes the benchmark's failure taxonomy. The most prevalent implementation error is missing essential components (39.71%), followed by environment or dependency failures at execution time (29.38%) and script or file errors (23.84%). On the reasoning side, design failures include incomplete or misclassified variables (16.05%), and conclusion failures include missing content (26.18%) or incorrect interpretation (19.66%).

The pattern is important: the dominant bottlenecks are not only "getting the final number wrong." Agents frequently fail much earlier by omitting a necessary change, misconfiguring the environment, or running the wrong procedure. This supports the benchmark's decision to separate D, I, E, and C rather than scoring only a final outcome.

6.4 Cost, time, and metric sensitivity

The benchmark artifacts include a cost-time analysis (Figure 5b) showing that higher spending or longer runtimes do not automatically translate into strong end-to-end performance. This matters for evaluation because some agents can appear competitive by exploring longer or using more expensive models, yet still fail on conjunctive correctness. EXP-Bench therefore encourages reporting both resource usage and strict outcome-based metrics.

7 Discussion

EXP-Bench contributes an evaluation framework for a specific capability: carrying out paper-grounded AI research experiments with enough rigor that design, code, execution, and conclusion agree. That is a meaningful target for research-agent evaluation, but it is narrower than scientific discovery as a whole. High performance on EXP-Bench would not imply broad scientific autonomy, and low performance does not show that agents are useless for research assistance. The benchmark measures a concrete operational slice of experimentation.

The benchmark also has clear limitations. First, it is built from 51 papers from two 2024 venues, so it should not be treated as exhaustive coverage of AI research practice. Second, D, I, and C rely on an LLM judge; the included stability analysis helps, but judged scores remain less direct than execution. Third, masking and monitoring reduce shortcutting and contamination risk, but cannot eliminate those risks completely. Fourth, the available benchmark artifacts emphasize execution-based validation and human review rather than formal inter-annotator agreement, so users should interpret judged components accordingly. This distinction affects how results should be reported. Improvements in D or C alone are informative for debugging agent behavior, but evidence for automated experimentation is much stronger when gains also appear on I·E and All·E✓, where the claimed conclusion is tied to an actually executed run. We therefore view stagewise and conjunctive reporting as complementary: the former localizes errors, and the latter tests whether a system can close the loop.

Within those bounds, EXP-Bench is useful because it reveals where current systems fail. The results suggest that progress on local code editing or agent scaffolding alone is insufficient. Reliable AI experimentation appears to require better decomposition of experimental variables, more robust environment handling, tighter coupling between implementation and interpretation, and evaluation procedures that penalize broken end-to-end chains rather than rewarding partial plausibility.

8 Conclusion

This paper presented EXP-Bench, a benchmark for evaluating whether AI agents can conduct complete AI research experiments grounded in real papers and repositories. EXP-Bench combines semi-automated task extraction, execution-based verification, integrity monitoring, and stagewise plus conjunctive metrics across 461 tasks from 51 recent AI papers. Empirically, current agents demonstrate partial competence on design, coding, and interpretation, but almost never complete the full workflow reliably once execution and conclusion consistency are required. EXP-Bench is therefore most valuable not as a leaderboard for declaring the problem solved, but as an execution-grounded instrument for measuring where research agents succeed, where they break, and what kinds of progress would actually matter.

References

[1] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Mądry. Mle-bench: Evaluating machine learning agents on machine learning engineering, 2024.

[2] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, et al. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. *arXiv preprint arXiv:2410.05080*, 2024.

[3] Q. Huang, J. Vora, P. Liang, and J. Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation, 2024.

[4] P. T. J. Kon, J. Liu, Q. Ding, Y. Qiu, Z. Yang, Y. Huang, J. Srinivasa, M. Lee, M. Chowdhury, and A. Chen. Curie: Toward rigorous and automated scientific experimentation with ai agents, 2025.

[5] R. Li, T. Patel, Q. Wang, and X. Du. Mlr-copilot: Autonomous machine learning research based on large language models agents. *arXiv preprint arXiv:2408.14033*, 2024.

[6] S. Li, Z. Wang, Z. Liu, C. Tan, H. Lin, D. Wu, Z. Chen, J. Zheng, and S. Z. Li. Moganet: Multi-order gated aggregation network, 2024.

[7] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. Agentbench: Evaluating llms as agents, 2023.

[8] G. Starace, O. Jaffe, D. Sherburn, J. Aung, J. S. Chan, L. Maksin, R. Dias, E. Mays, B. Kinsella, W. Thompson, et al. Paperbench: Evaluating ai's ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.

[9] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[10] W.-H. Weng et al. Towards an ai co-scientist, 2025.